

8. To build a basic Convolutional Neural Network (CNN) model using the MNIST dataset.

Build a Basic Convolutional Neural Network (CNN) Model Using the MNIST Dataset

Aim

To build and train a basic Convolutional Neural Network (CNN) model using the MNIST handwritten digit dataset for image classification.

Introduction

A Convolutional Neural Network (CNN) is a deep learning algorithm designed for image processing and computer vision tasks. Unlike traditional neural networks, CNNs automatically extract important features such as edges, textures, and shapes from images.

CNNs are widely used in handwritten digit recognition, face recognition, object detection, medical image analysis, and autonomous driving.

CNN Architecture

A basic CNN consists of the following layers:

- Input Layer
- Convolution Layer (Conv2D)
- ReLU Activation Function
- Max Pooling Layer
- Flatten Layer
- Dense (Fully Connected) Layer
- Output Layer (Softmax)

MNIST Dataset

The MNIST dataset is one of the most popular datasets used for handwritten digit recognition.

Dataset Information

- Total Images: 70,000
- Training Images: 60,000
- Testing Images: 10,000
- Image Size: 28 × 28 pixels
- Number of Classes: 10 (Digits 0–9)

MNIST Dataset

The MNIST dataset is one of the most popular datasets used for handwritten digit recognition.

Dataset Information

- Total Images: 70,000
- Training Images: 60,000
- Testing Images: 10,000
- Image Size: 28 × 28 pixels
- Number of Classes: 10 (Digits 0–9)
- Image Type: Grayscale

Algorithm

1. Import the required libraries.
2. Load the MNIST dataset.
3. Display sample images.
4. Normalize pixel values.
5. Reshape the images.
6. Build the CNN model.
7. Display the model summary.
8. Compile the model.
9. Train the model.
10. Evaluate the model.
11. Predict handwritten digits.
12. Display prediction results.

```
In [2]: #Import Libraries
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras import datasets, layers, models
```

```
In [3]: # Load MNIST Dataset

(X_train, y_train), (X_test, y_test) = datasets.mnist.load_data()

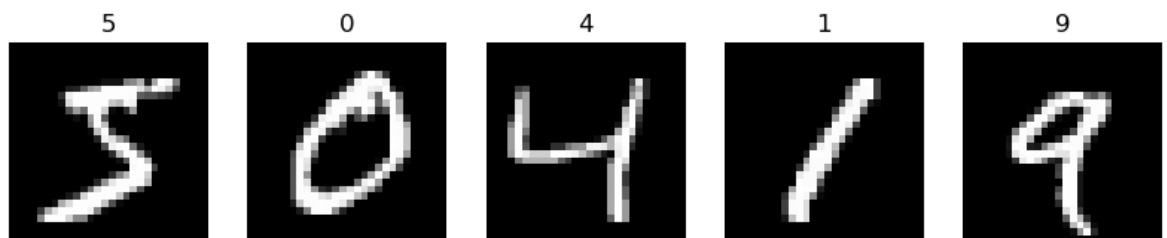
print("Training Images :", X_train.shape)
print("Testing Images :", X_test.shape)
```

```
Training Images : (60000, 28, 28)
Testing Images : (10000, 28, 28)
```

```
In [4]: #Display Sample Images
plt.figure(figsize=(10,4))

for i in range(5):
    plt.subplot(1,5,i+1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title(y_train[i])
    plt.axis('off')

plt.show()
```



```
In [5]: #Normalize Data
# Normalize pixel values

X_train = X_train / 255.0
X_test = X_test / 255.0
```

```
In [6]: # Reshape images

X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)

print(X_train.shape)

(60000, 28, 28, 1)
```

Building the CNN Model

The CNN model consists of:

- Convolution Layer (32 Filters)
- Max Pooling Layer
- Flatten Layer
- Dense Layer (128 Neurons)
- Output Layer (10 Neurons with Softmax)

```
In [7]: #Build CNN Model
model = models.Sequential()

model.add(layers.Conv2D(
    filters=32,
    kernel_size=(3,3),
    activation='relu',
    input_shape=(28,28,1)
))

model.add(layers.MaxPooling2D(pool_size=(2,2)))

model.add(layers.Flatten())

model.add(layers.Dense(128, activation='relu'))

model.add(layers.Dense(10, activation='softmax'))
```

```
In [8]: #Model Summary
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	
conv2d_1 (Conv2D)	(None, 26, 26, 32)	
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)	
flatten_1 (Flatten)	(None, 5408)	
dense_2 (Dense)	(None, 128)	
dense_3 (Dense)	(None, 10)	



Total params: 693,962 (2.65 MB)

Trainable params: 693,962 (2.65 MB)

Non-trainable params: 0 (0.00 B)

```
In [9]: #Compile Model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
In [10]: #Train Model
history = model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)
```

```
Epoch 1/5
750/750 ————— 21s 23ms/step - accuracy: 0.8764 - loss: 0.4164
- val_accuracy: 0.9770 - val_loss: 0.0838
Epoch 2/5
750/750 ————— 16s 21ms/step - accuracy: 0.9780 - loss: 0.0738
- val_accuracy: 0.9817 - val_loss: 0.0635
Epoch 3/5
750/750 ————— 16s 22ms/step - accuracy: 0.9858 - loss: 0.0473
- val_accuracy: 0.9830 - val_loss: 0.0623
Epoch 4/5
750/750 ————— 18s 24ms/step - accuracy: 0.9912 - loss: 0.0295
- val_accuracy: 0.9854 - val_loss: 0.0484
Epoch 5/5
750/750 ————— 17s 23ms/step - accuracy: 0.9943 - loss: 0.0196
- val_accuracy: 0.9842 - val_loss: 0.0561
```

```
In [11]: #Evaluate Model
test_loss, test_accuracy = model.evaluate(X_test, y_test)

print("Test Loss :", test_loss)
print("Test Accuracy :", test_accuracy)
```

```
313/313 ————— 2s 5ms/step - accuracy: 0.9825 - loss: 0.0585
Test Loss : 0.04850538820028305
Test Accuracy : 0.9851999878883362
```

```
In [12]: #Predict Digits
predictions = model.predict(X_test)

predicted_digit = predictions[0].argmax()

print("Predicted Digit :", predicted_digit)
print("Actual Digit :", y_test[0])
```

```
313/313 ————— 2s 6ms/step
Predicted Digit : 7
Actual Digit : 7
```

```
In [13]: #Display Prediction
plt.figure(figsize=(4,4))

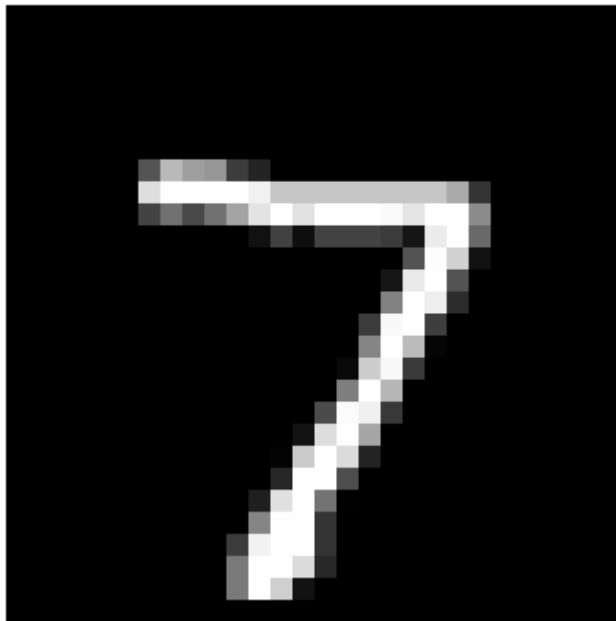
plt.imshow(X_test[0].reshape(28,28), cmap='gray')

plt.title(f"Predicted : {predicted_digit} | Actual : {y_test[0]}")

plt.axis('off')

plt.show()
```

Predicted : 7 | Actual : 7



Expected Output

- MNIST dataset is loaded successfully.
- Five sample handwritten digit images are displayed.
- The CNN model summary is displayed.
- The model trains successfully for 5 epochs.
- Test accuracy is approximately **98%–99%**.
- The predicted digit and actual digit are displayed along with the corresponding test image.

Observation

- The CNN automatically extracts features from handwritten digit images.
- The model learns image patterns effectively using convolution and pooling layers.
- The trained model achieves high accuracy on the MNIST test dataset.

Result

A basic Convolutional Neural Network (CNN) model was successfully built and trained using the MNIST dataset. The model accurately classified handwritten digits with a test accuracy of approximately **98%–99%**.

